ELSEVIER

Robotics and Autonomous Systems 36 (2001) 103–119

Robotics and Autonomous Systems

www.elsevier.com/locate/robot

Vision-based localization for mobile robots

Giovanni Adorni [a,*], Stefano Cagnoni [b], Stefan Enderle [c], Gerhard K. Kraetzschmar [c], Monica Mordonini [b], Michael Plagge [d], Marcus Ritter [c], Stefan Sablatnög [c], Andreas Zell [d]

[a] *Department of Communications, Computer and System Science, University of Genoa, Via Opera Pia 13, I-16145 Genoa, Italy*
[b] *Department of Computer Engineering, University of Parma, Parco Area delle Scienze 181a, I-43100 Parma, Italy*
[c] *Neural Information Processing Department, University of Ulm, Oberer Eselsberg, D-89069 Ulm, Germany*
[d] *Department of Computer Architecture, University of Tübingen, Sand 1, D-72076 Tübingen, Germany*

Abstract

Robust self-localization and repositioning strategies are essential capabilities for robots operating in highly dynamic environments. Environments are considered highly dynamic, if objects of interest move continuously and quickly, and if chances of hitting or getting hit by other robots are quite significant. An outstanding example for such an environment is provided by RoboCup.

Vision system designs for robots used in RoboCup are based on several approaches, aimed at fitting both the mechanical characteristics of the players and the strategies and operations that the different roles or playing situations may require. This paper reviews three approaches to vision-based self-localization used in the RoboCup middle-size league competition and describes the results they achieve in the robot soccer environment for which they have been designed. © 2001 Elsevier Science B.V. All rights reserved.

Keywords: Self-localization; Visual features; Dynamic environments; RoboCup

1. Introduction

To perform complex navigation tasks and to coordinate its movement with other agents in indoor environments, an autonomous mobile robot needs knowledge about the world in which it is moving. This enables it to recognize the main landmarks and determine its position with respect to the environment. In fact, answering the question "Where am I?" is one of the most crucial and, often, critical tasks (both in terms of quality and time complexity) for an autonomous mobile robot [6,12]. The self-localization problem is usually solved with a combination of relative and absolute position measurement techniques [7]. Odometry and inertial navigation (dead reckoning) systems are usually integrated with methods employing active beacons, natural or artificial landmarks as well as environment model matching, in order to compensate for wheel slippage and other positioning errors [13]. Natural and artificial landmarks are distinct features that a robot can recognize from its sensory input. In general, landmarks have a fixed

* Corresponding author.
E-mail addresses: adorni@dist.unige.it (G. Adorni), cagnoni@ce.unipr.it (S. Cagnoni), steve@neuro.informatik.uni-ulm.de (S. Enderle), gkk@neuro.informatik.uni-ulm.de (G.K. Kraetzschmar), monica@ce.unipr.it (M. Mordonini), plagge@informatik.uni-tuebingen.de (M. Plagge), ritter@wonderbits.de (M. Ritter), stefan@neuro.informatik.uni-ulm.de (S. Sablatnög), zell@informatik.uni-tuebingen.de (A. Zell).

0921-8890/01/$ – see front matter © 2001 Elsevier Science B.V. All rights reserved.
PII: S0921-8890(01)00138-5

well-known position within an absolute reference system, relative to which a robot can localize itself. The main task in self-localization is then to recognize such landmarks reliably by processing the sensory input to compute the robot's position [4,26,30].

The literature concerning computer vision reports on a large number of natural and artificial landmark positioning systems integrated in autonomous robot architectures [3,8]. Several authors have tried to find a good correlation between what the robot perceives through a camera and its internal representation of the environment [21,27]. In indoor environments, edges of doors or walls are commonly used as landmarks to perform this correlation, but often such patterns are too generic to allow an accurate localization in a more complex environment. Researchers have often focused on the choice of *optimal* shapes or patterns as landmarks so that robust localization can be obtained from a single [23,24] or from multiple images [20]. Furthermore, considerable work has been devoted to the treatment of error in sensor observations [5]. Sometimes it can be useful to evaluate alternative techniques that can recognize landmarks which can be specially designed to carry both human- and robot-oriented visual information, though they might also be complemented by other *natural* signs which are often found in office and industrial environments [2,28]. Often these methods do not yield high localization accuracy, but there is a number of practical situations, e.g., docking and navigating in small openings, where coarser robot positioning accuracy suffices.

The RoboCup environment provides researchers with a common environment in which it is possible to test and compare different visual self-localization methods reliably. Self-localization in RoboCup is very important to achieve coordination between players, both to defend the team's penalty area and to make goal-oriented decisions [19]. Although the world of RoboCup is simpler than many real-life indoor environments, it is a very dynamic world in which the robot has to react quickly to the changes caused by the continuous movement of the robots and the ball. Most robots rely on odometry for real-time self-localization. However, odometry fails immediately when a robot hits other robots. The appeal of the RoboCup challenge is actually due to its highly dynamic environment, in which robots usually obtain only partial views of the scene, since reference markers are often occluded by other robots.

To reposition themselves, robots in RoboCup use different kinds of sensors, such as cameras, laser range finders, sonars, and so on. While self-localization based on laser range finders provides a very accurate and robust estimation of robot position, visual sensors are more flexible; the number and configuration of cameras can be chosen according to which other tasks are performed by the vision system and to the role that the robot has within the team. Most distance sensors require that the field of play be surrounded by walls. This limits their future use, when RoboCup eventually moves towards a more natural, wall-free environment.

In RoboCup, according to the current rules, the appearance of relevant objects is well known and it is therefore possible to provide robots with some geometric models of the field or of its subparts. Visual self-localization is typically performed by matching and comparing the robot's view of the world with a model. This comparison can often be used to refine and compensate for the errors introduced by odometric data, but in some cases a robot must be able to localize itself without any help from sensory systems other than that of vision.

Several kinds of vision systems have been applied in RoboCup. While most robots use traditional single-camera vision systems, it is becoming quite common to adopt more complex solutions. For example, the problem of having only a limited field of view can be solved using a camera pointing upwards to a concave mirror to obtain a global vision system. Sometimes, different vision system designs and localization strategies are used for robots playing on the same team, which allows for more specialized robots to be built according to their roles but makes sharing data among robots more difficult.

This paper reviews three approaches to visual self-localization used in the RoboCup middle-size (F-2000) competition, describing and comparing the results they achieve in the RoboCup test-bed environment. The paper is organized as follows: a self-localization method based on an omnidirectional vision system is illustrated in Section 2, a self-localization based only on the comparison between a pair of images from a binocular vision system is proposed in Section 3, and a self-localization based on the Monte Carlo localization technique is

described in Section 4. Section 5 reports some results of the above described techniques. Finally some conclusions from which possible future work can stem are drawn in Section 6.

2. Robust vision-based localization with monocular vision

A lot of successful work on self-localization in indoor or bounded environments with laser range finders has been presented in the past. As already mentioned in Section 1, these methods are very accurate, fast and robust. The main drawback in the use of laser range finders or more generally in the use of data from exclusive distance-measuring sensors is the almost complete lack of information about the objects from which the distance measurements originate. Only by extracting geometric features such as edges or lines from the data, information about objects in the environment or the environment itself can be gathered. This leads for example to the inability to perform a global self-localization task even in rather simple periodic or symmetric environments like the RoboCup, where there are always at least two equivalent position estimates with a heading difference of π. A further drawback which affects especially 2D laser range finders is the restriction of the effective range to a 2D plane. Any relevant object must have an intersection with this plane to enable the laser range finder to detect it.

In this section, we present an algorithm which combines the advantages of approaches based on laser range finders with the flexibility provided by a vision-based approach. For this purpose, an omnidirectional camera was built, which is able to provide both object recognition and distance measurement capabilities simultaneously. The design of this camera was developed by Franz [17] from an earlier MPI design used for biologically inspired vision experiments. A major drawback of the proposed method is the poor resolution for larger distances, which may compromise robustness. To overcome this problem, a set of simple algorithms were implemented to verify and enhance the position estimates of the main algorithm. The algorithm works particularly well in an area with good visual clues and, hence, is especially suited for a robot goalkeeper.

2.1. Localization by using distance data from image processing

The 360° camera supplies a circular image of the environment, which is sampled radially from the center of the circle to the fringe (see Fig. 1). This is done for 360 adjacent radii, which results in a spatial resolution of 1°. Each sampled pixel on a line is assigned, according to its color, to one of the following color classes: *GOAL-YELLOW*, *GOAL-BLUE*, *FIELD*, *WALL*, *BALL*, *ROBOT*, *UNKNOWN*. By using the class information of pixels along a radial line representing the area at the horizon, the type of the static

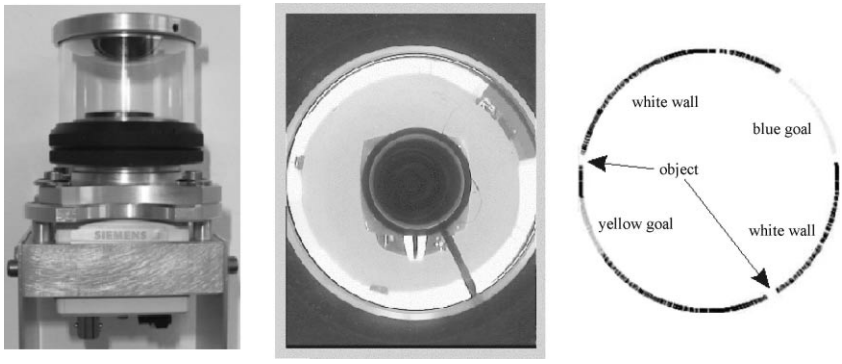


Fig. 1. The left-hand picture shows the construction of the 360° camera. The middle picture shows an image directly taken by the camera. The blue goal is in the upper right, the yellow goal in the lower left corner of the image. The rightmost picture shows the outcome of the type classification of the circular horizon image.

environment (*GOAL-YELLOW*, *GOAL-BLUE*, *WALL*) under a certain direction can be determined. This detection is quite stable, since several points around the horizon are taken into account.

By specifying the mapping between a pixel on a radial line of the image and the distance to the corresponding point on the field in the real environment, it is possible to provide a vision-based distance measurement with the 360° camera. Strictly speaking, only distances to points on the field can be measured, while distances to an arbitrary point, e.g. to an object within the field, cannot be measured. To obtain the mapping between pixels in the image and distances on the field a calibration is needed. Since this method of determining distances is based on the detection of a single point type transition along a radial line in the image, it is sensitive to changing lighting conditions or badly trained object colors for classification and is therefore error-prone and subject to noise. This is usually not true for the calibration process itself, which can be performed under almost ideal conditions.

Now the same techniques for self-localization as used with laser scanner data can be applied [6,33]. But in contrast to laser scanner data, which have a nearly constant error and resolution independent of the measured distances, the error in the data from the vision system increases with distance (see Fig. 9) while resolution decreases. This raises the need for further processing of a position estimate. We will present a method which can perform a verification of a given position estimate on the basis of the more stable horizon type detection [32].

Let us assume that the goalkeeper defends the blue goal. First the self-localization algorithm type-filters the data from the image processing. Only distance data from directions under which the blue goal is visible at the horizon are used for further processing. Next these type-filtered distances are averaged with the values from adjacent directions within a certain angle to reduce the high-frequency noise within the data. Now the algorithm tries to fit a line to the data representing the blue goal. Because the exact position of this goal is known in the global coordinate frame the algorithm can calculate the robot position and orientation.

For the verification of the given position estimate, a criterion must be found on which a decision can be made, whether the estimate is appropriate or not. For that the algorithm falls back on the robust detection of the environment. By clustering the data with the same type a certain viewing angle α for the blue goal can be quantified (see Fig. 2). Assuming that the origin of the global coordinate frame falls on the middle of the goal line it can be shown that the robot position must be somewhere on a circle with radius r and origin (x_0, y_0), where $r = b/(2\sin(\alpha))$ and $x_0 = 0$, $y_0 = b/(2\tan(\alpha))$, b being the width of the goal. So if an estimated position falls on the circle described by this equation the algorithm assumes the correctness of the estimate.

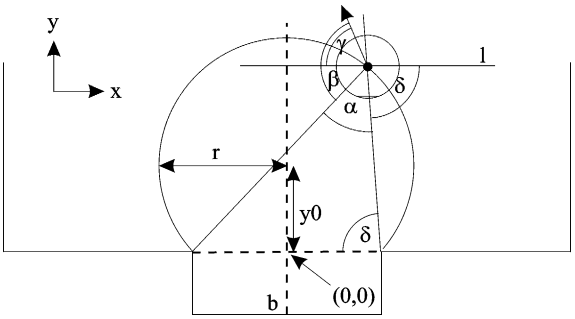


Fig. 2. If the goal is seen under an angle α, the robot position falls on a circle with origin $(0, y_0)$ and a radius r. b is the width of the goal. The origin of the global coordinate frame is at the intersection of the dashed lines. β is the angle between the reference direction for the robot heading and the start of the cluster indicating the goal. γ is the angle between the robot global heading (e.g. from compass) and the known orientation (from an a priori model of the environment) of the goal line. Line l is parallel to the goal line.

This algorithm for self-localization may fail under certain conditions, e.g. if the distance to the visual clues in the environment is too large for a reliable distance measurement. Therefore two other methods are derived to estimate the global position as backup algorithms. Both are only usable under certain and differing conditions.

2.2. Localization by using global heading data

If not only the viewing angle α of the own goal but also the heading of the robot with respect to a global coordinate frame is known (e.g. by compass data), the position of the robot with respect to this coordinate frame can be calculated (see Fig. 2). The position is at the intersection of the previously described circle and

a line:

$$x^2 + \left(y - \frac{b}{2\tan(\alpha)}\right)^2 = \left(\frac{b}{2\sin(\alpha)}\right)^2 \quad \text{and}$$

$$y = \tan(\pi - \delta)\left(x - \frac{b}{2}\right). \tag{1}$$

δ can be calculated as $\delta = \pi - (\beta + \alpha) + \gamma$ where β is the angle between the reference direction for the robot heading and the start of the domain indicating the goal and γ the angle between the robot global heading and the known orientation of the goal line.

2.3. Localization by using data from odometry

While the robot is moving, the viewing angle $\alpha_{1,2}$ and the angle $\beta_{1,2}$ between the reference direction for the robot heading and the beginning of the area indicating the goal can be recorded at two different positions t_1 and t_2 along with the distance d traveled between these two positions. From these data the position of the robot with respect to a global coordinate frame can be calculated (see Fig. 3).

Such a position is determined by the intersection of two circles centered on the edges of the goal line, having radii of l_1 and l_2, respectively. As in the previous section, the coordinates of the edges within the global coordinate frame are given by the a priori model of the environment. The radii l_1 and l_2 can be calculated as

$$l_1 = \frac{d\sin(\beta_1)}{\sin(\beta_2 - \beta_1)} \quad \text{and}$$

$$l_2 = \frac{d\sin(\beta_1 + \alpha_1)}{\sin(\beta_2 - \beta_1 + \alpha_2 - \alpha_1)}. \tag{2}$$

This algorithm could easily be used not only with translating but also with rotating movements of the robot. In our case this is not reasonable, due to the skid drive of the goalkeeper robot, which leads to unreliable odometry data when the robot is turning. The need for the robot to move before this algorithm can be applied is its main drawback.

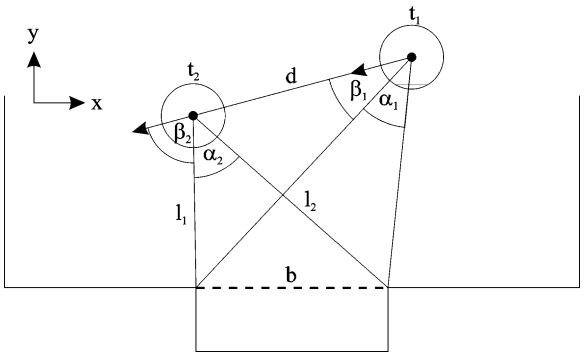


Fig. 3. Moving the robot from t_1 to t_2, where the distance between t_1 and t_2 is d, leads to a change of the opening angle α_1 to α_2 and to a change of the direction angle β_1 to β_2. l_1 and l_2 are the distances to the two goal posts.

3. Binocular vision and geometric reasoning

Unlike the method described in the previous section, that relies on data coming from an omnidirectional vision sensor, the self-localization method described in this section has been designed for use on a robot goalkeeper supplied with a vision system based on two wide-angle cameras. The fields of view of the two cameras overlap in a region of about 20° which corresponds to the center of the zone that lies immediately in front of the robot. This configuration makes it possible to obtain a visual field about 220° wide (see Fig. 4). In operating conditions, when the ball is not detected by either of the two cameras, the camera from which the image is acquired is switched at each new frame acquisition. When the ball is detected by one of the two cameras, that camera keeps acquiring images until the ball enters the overlap region. At that point the other camera is switched on for acquisition.

The goalkeeper software architecture consists of a multiagent system composed of three agents: a *visual perception agent*, a *goalkeeper agent*, and a *cooperative agent*.

The visual perception agent's duties are to detect and localize the objects that are in the playing environment and to make the robot aware of its position inside the field. The visual perception agent therefore is essential to three main robot behaviors: object detection and localization, absolute self-localization, which is described in detail in this section, and repositioning. The cooperative agent enables the robot to 'talk', using a messaging system, with the other team members and with a remote system from which it receives



Fig. 4. The goalie's binocular vision system.

a start signal at the beginning of each half. Finally, the goalkeeper agent plans the robot's behavior according to the activity of the other two agents. In particular, it drives the engines and actuators of the robot, and activates the robot behaviors in the different phases of the game.

The goalie's role and position inside the field strongly influence and constrain its mechanical design and its motion strategy: its movements should not extend beyond the goal boundaries, since it has to stand in front of it to defend it. A RoboCup goalie's motion control strategy has to satisfy two often counteracting needs. On the one hand, the robot must react promptly to sudden changes, i.e., it must exhibit a mostly reactive behavior. The ball speed is relatively high and most computation time has to be dedicated to image processing tasks. For this reason, the specifications for the algorithms implemented in the goalkeeper agent are that the input data, which encode ball position, speed and direction, must be concise and output commands must be as direct and immediately executable as possible. On the other hand, the goalkeeper agent must keep the robot in front of its goal and well centered with respect to the direction from which the ball is most likely to come, at any moment of a game. To do so, the robot needs to be aware not only of its position with respect to the field but also of its orientation with respect to the goal line. Data from the odometers are too unreliable for this purpose. Frequent contacts with other robots in the penalty area and the rapid movements needed to follow ball deviations can make wheels skid and cause "false movements" to be recorded. The landmark-based self-localization method, implemented in the visual perception agent, has been designed to overcome such a drawback.

The actions performed by the goalkeeper agent are hierarchically organized at different level of priority. The highest priority is attributed to "active defense" actions, i.e., those performed to intercept or control the ball. During such actions, the visual perception agent processes images to detect the ball. The other actions, preparatory to the "active" robot behaviors, including the self-positioning maneuver, are cyclically performed during "inactive" game periods, that is when the ball is out of the goalie's region of operation.

The self-localization process uses the white lines of the penalty area as artificial landmarks and draws its decisions by comparing the two views coming from the left and right cameras of the binocular system. Such a comparison makes it possible to compute the actual robot position inside the penalty area, and to plan and perform the right repositioning maneuver when needed. When the self-localization process is running, the image pair received from the two cameras is first processed to detect the lines delimiting the penalty area. Segmentation is performed by a two-step algorithm. In the first step, color segmentation, the image depth is reduced by labeling each pixel as belonging to one of six classes, corresponding to the six possible colors that can be found in the RoboCup environment. This is a pre-processing step that is always performed after image acquisition, whatever section of the code of the goalkeeper agent is going to use it. In the second step, a simple edge extraction filter is applied, whose results are then AND-ed with the binary output that is obtained by setting all non-white pixels to 0. Doing so, only the edges of the white areas can be extracted and the parameters of the straight line equations can be computed. The limited number of pixel involved and its possible real-time implementation [18] make the Hough transform appropriate to perform such a task. Moreover, the Hough transform results are expressed in polar coordinates, allowing for an easy computation of the line equations.

The parameters of the lines obtained as described above allow the computation of three features that are compared with a set of references that correspond to a situation in which the robot is in its "default position", that is standing in the middle of the penalty area, in front of the goal at a preset distance and parallel to the front line. Such features are the coordinates of the point P_i where the two views of the front line of the penalty area apparently intersect, and the apparent angle α between the two views of the lines. If the goalie is parallel to the front line of the penalty area, then the left and right views of the line are symmetric (as, for example, in Fig. 4). If the goalie is rotated and/or shifted sidewards with respect to the middle of the penalty area, such symmetry is lost. The analysis of the asymmetry of the two views of the front line makes it possible to estimate the robot's absolute position with respect to the penalty area and, thus, to the reference coordinate system of the playing field.

The four cases shown in Fig. 5 illustrate how geometric reasoning is performed. The upper parts of the four figures show the views from the two cameras as acquired when the robot is in the default position and in three different displaced positions inside the penalty area. In the lower parts of the figures, the corresponding diagrammatic representation of the views is reported. Such figures report the results of line segmentation as in a reference system having the

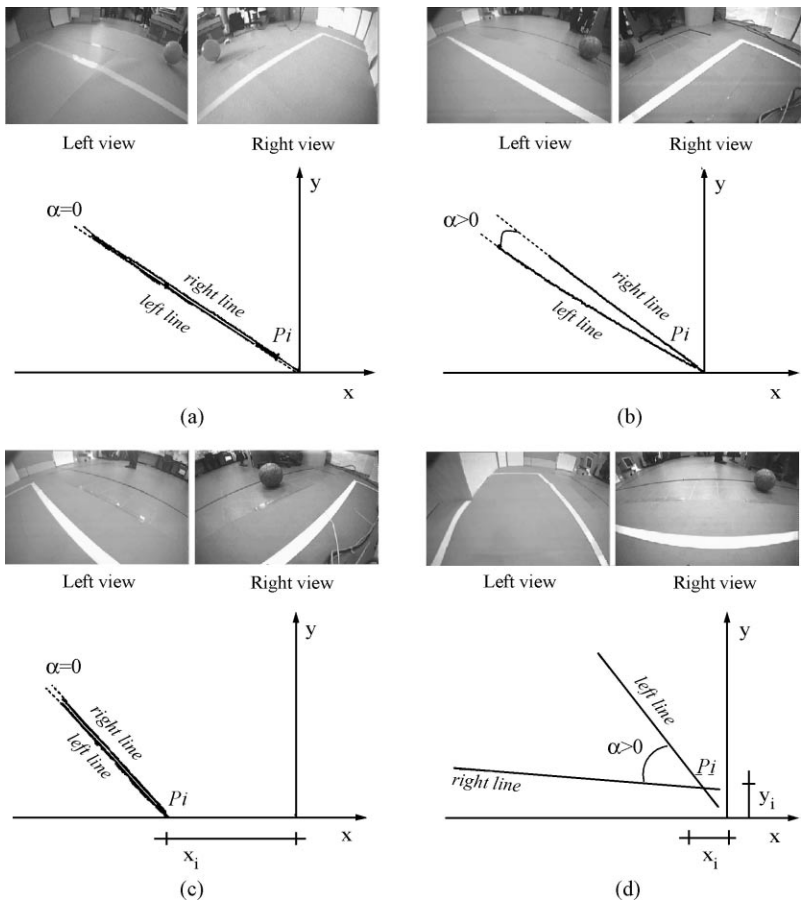


Fig. 5. Penalty area view as seen by the robot when in a barycentric position parallel to the reference line (default position) (a), in the center of the penalty area in front of the right post (b), standing on the front line, centered with respect to the goal (c), turned left through about 45° (d). Below each view is the corresponding diagrammatic representation.

y-axis corresponding to the median axis of the area in which the fields of view of the two cameras overlap, and the x-axis coincident with the bottom of the frames. A "horizontal flip" operation is applied to the view of the line taken from the right camera and the two views are superimposed to simplify the computation and the visualization of the parameters $P_i(x_i, y_i)$ and α.

When the robot is parallel to the front line, centered on the median vertical axis of the penalty area, P_i lies on the x- or y-axis, $\alpha \simeq 0$, as shown in Fig. 5(a) and (c). When the robot moves forward, x_i becomes negative (with $y_i = 0$), as shown in Fig. 5(c); when it moves backwards, y_i becomes positive (with $x_i = 0$). A lateral shift of the robot results in a non-zero value for α (Fig. 5(a) and (b)). Such a small divergence is due to the distortion effects introduced by the cameras, but it is sufficient to compute the lateral shift, to which it is roughly proportional. Any rotation of the robot results in a composition of the previously described conditions. Fig. 5(d) shows an example in which the robot is rotated through about 45°.

4. Monocular vision and vision-based Monte Carlo localization

Monte Carlo localization (MCL) has recently become a very popular framework for solving the self-localization problem in mobile robots [15,16]. This method is very reliable and robust with respect to noise, especially if the robots are equipped with laser range finders or sonar sensors. However, in environments like the popular RoboCup domain [22], providing a laser scanner for each robot is difficult or impossible and sonar data is extremely noisy due to the highly dynamic environment. Furthermore, distance sensors will ultimately be of little value, because at some time in the future RoboCup fields will not have walls any more. Thus, a vision-based MCL approach exploiting visual features extracted from the robot's unidirectional camera is of great interest.

4.1. Markov localization

The theoretical foundation for MCL is provided by Markov localization [15], where the position of a robot is estimated by computing the probability distribution $\mathrm{Bel}(l)$ over all possible positions $l = \langle x, y, \theta \rangle$ in the environment. The Markov algorithm starts with an initial distribution $\mathrm{Bel}(l)$ that depends on the initial knowledge of the robot's position. As the robot operates, two kinds of update steps are iteratively applied to incrementally refine $\mathrm{Bel}(l)$:

1. *Belief projection across robot motion*. When the robot moves, the belief about its position is projected across actions. A *motion model* $P(l|l', m)$ is used to account for behavioral uncertainty and to predict the likelihood of the robot being in position l when the robot executed motion command m and was previously in position l'. Here, it is assumed that the new position depends only on the previous position and the movement (Markov property). Using the motion model, the distribution of the robot's position belief $\mathrm{Bel}(l)$ can be updated according to the commonly used formula for Markov chains [9]: $\mathrm{Bel}(l) = \int P(l|l', m)\,\mathrm{Bel}(l')\,\mathrm{d}l'$.
2. *Integration of sensor input*. The data obtained from the robot's sensors are used to update the belief $\mathrm{Bel}(l)$. For this step, an *observation model* $P(o|l')$ must be provided which models the likelihood of an observation o (a sensor reading) given the robot is at position l'. $\mathrm{Bel}(l)$ is then updated by applying Bayes' rule as follows: $\mathrm{Bel}(l) = \alpha\, P(o|l')\,\mathrm{Bel}(l')$, where α is a normalization factor ensuring that $\mathrm{Bel}(l)$ integrates to 1.

Markov localization provides a mathematical framework for solving the localization problem. Unlike methods based on Kalman filtering [25], it lends itself easily to multimodal distributions. However, implementing Markov localization on a mobile robot efficiently is a non-trivial task.

4.2. Monte Carlo localization

MCL [16] is an efficient implementation of the Markov localization approach (see, for example [15]). The continuous and infinite probability distribution $\mathrm{Bel}(l)$ is represented by a discrete set of N samples $S = \{s_1, \ldots, s_N\}$. Each sample $s_i = \langle l_i, p_i \rangle$ consists of a robot *position* l_i and a *weight* p_i. The weight corresponds to the likelihood of l_i being the robot's correct position: $p_i \approx \mathrm{Bel}(l_i)$ [31]. Furthermore, we assume $\sum_{i=1}^{N} p_i = 1$.

The algorithm for MCL is adopted from the general Markov localization framework. Initially, a set of samples reflecting a priori knowledge about the robot's position is generated. During robot operation, the following two kinds of update steps are iteratively executed:

1. *Sample projection across robot motion*. A new sample set S is generated from a previous set S' by applying the motion model. For each sample $\langle l', p' \rangle \in S'$, a new sample $\langle l, p' \rangle$ is added to S, where l is randomly drawn from the density $P(l|l', m)$, which takes into account the robot's motion uncertainties like drift and translational and rotational errors.
2. *Observation update and weighted resampling*. All samples are re-weighted by incorporating the sensor data o and applying the observation model $P(o|l')$.

   Given a sample $\langle l', p' \rangle$, the new weight p is given by $p = \alpha P(o|l')p'$. The weights for the samples in S' provide a probability distribution, which is used to construct a new sample set S by randomly drawing samples from S' using the distribution given by the weights.

MCL has several interesting properties; for instance, it can be implemented as an anytime algorithm [10], and it allows one to dynamically adapt the sample size (see [15] for a detailed discussion of MCL properties).

4.3. Vision-based Monte Carlo localization

There are mainly three cases where MCL based on distance sensor readings cannot be applied: (i) if distance sensors such as laser range finders or sonars are not available; (ii) if the readings obtained by these sensors are too noisy and unreliable, e.g. in a highly dynamic environment; (iii) if distance sensors are useless for self-localization, e.g. an open field with no or few objects detectable by the sensors. In all these cases, other sensory information must be used for localization. In RoboCup, a natural candidate is the visual channel, because many robots include cameras as standard equipment for object detection. An example for using visual information for MCL has been provided by Dellaert et al. [11]. They use information on an environment's lighting distribution, which is *continuously* obtained by a camera oriented towards the ceiling. However, an interesting question is whether the MCL approach still works when the number of observations is *sparse* and observations are *sporadic*, as is the case in our soccer robots.

4.3.1. Feature-based modeling

The sensor update mechanism needs a sensor model $P(o|l)$ which describes how likely a sensor reading o is at a given robot location l. This probability is derived from the distance $\text{dist}(o, \tilde{o})$ between the sensed measurement o and a measurement $\tilde{o}$, which is estimated from location l and an environment model. As it is not possible to efficiently estimate complete camera images and then compute image distances for hundreds of samples, we use a feature-based approach. After evaluating various feature detectors on our robots (see [14]), we decided to use the following features: (1) goal posts of the blue and yellow goals, (2) corners, and (3) distances to field edges. Taking into account the characteristics of the field and the robot's camera system, at most eight environment features can be detected at any time. The average number of detectable features is actually even lower.

4.3.2. Feature detection

In a first step the camera image is color-segmented in order to simplify the feature detection process. Based on the color-segmented image, we apply different kinds of filters to detect desired color discontinuities. The *goal post detector* detects a horizontal *white–blue* or a *white–yellow* transition for the blue or yellow goal post, respectively (see left image in Fig. 6). The *corner detector* searches for horizontal *green–white–green* transitions in the image (middle image in Fig. 6). The *distance estimator* estimates the distance to the field edges based on detected vertical *green–anything* transitions in the image. We select four specific columns in the image for detecting the field edges (rightmost image in Fig. 6).



Fig. 6. Detection of post, corner and edge features.

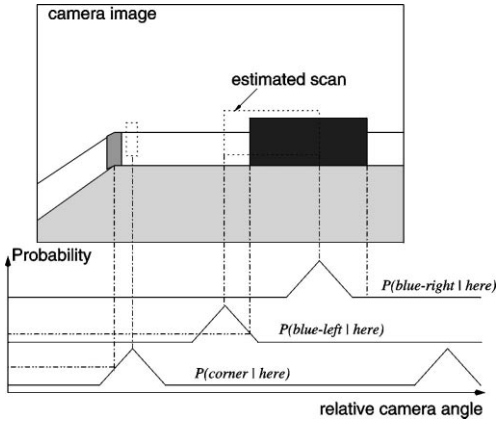


Fig. 7. Heuristics for estimating the probabilities $P(f_i|l)$.

4.3.3. Weight update

Let the sensor data o be a vector of n features $f_1 \cdots f_n$. If we assume that the detection of features depends solely on the robot's position and does not depend on the detectability of other features, then the features are independent and we can conclude $P(o|l') = P(f_1 \cdots f_n|l') = P(f_1|l') \cdots P(f_n|l')$. Note that n may vary, i.e., it is possible to deal with varying numbers of features. The sensor model $P(f_i|l)$ describes how likely it is to detect a particular feature f_i given a robot location l. In our implementation, this sensor model is computed by comparing the horizontal position of the detected feature with an estimated position as determined by a geometrical world model. The distance between the feature positions is mapped to a probability estimate by applying a heuristic function as illustrated in Fig. 7. The camera image shows three detectable features: a corner and two goal posts. The same features are expected, but estimated at slightly different positions. The application of these heuristics to the examples used in Fig. 6 are illustrated in Fig. 8. Probabilistic combination of evidence for several features yields significantly better results, as convincingly demonstrated by the rightmost image. Fig. 8 also illustrates various properties of our approach. The shape of the function causes all samples with comparatively high angular error to be drastically down-valued and successively being sorted out. Thus, detecting a single goal post will reshape the distribution of the sample set such that mostly locations that make it likely to see a goal post in a certain direction will survive in the sample set.

5. Evaluation and discussion

The algorithm described in Section 2.1 was implemented and evaluated on the goalkeeper robot of the Attempto team Tübingen.

Calibration. Fig. 9 presents a derived mapping of the position of pixels on a radial line in the camera image where the type classification changes (e.g. from *FIELD* to *WALL*) against the distance to this point in the environment. Two different methods are applied

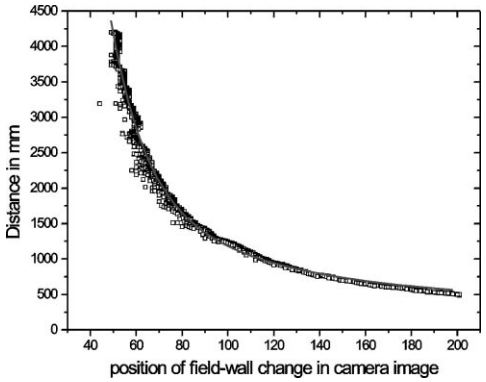


Fig. 9. The mapping of the position of a pixel on a radial line against the distance to the appropriate point on the field.

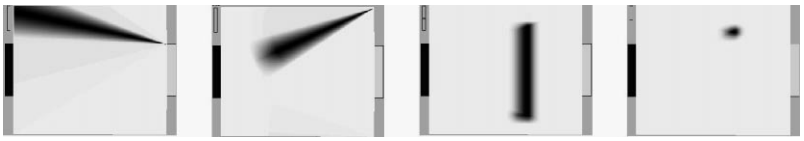


Fig. 8. Probability distributions $P(f_i|l)$ for all field positions l (with fixed orientation) given the detection of a particular feature f_i and feature combination. From left to right: goal posts, corners, distance measurements, and all three combined.

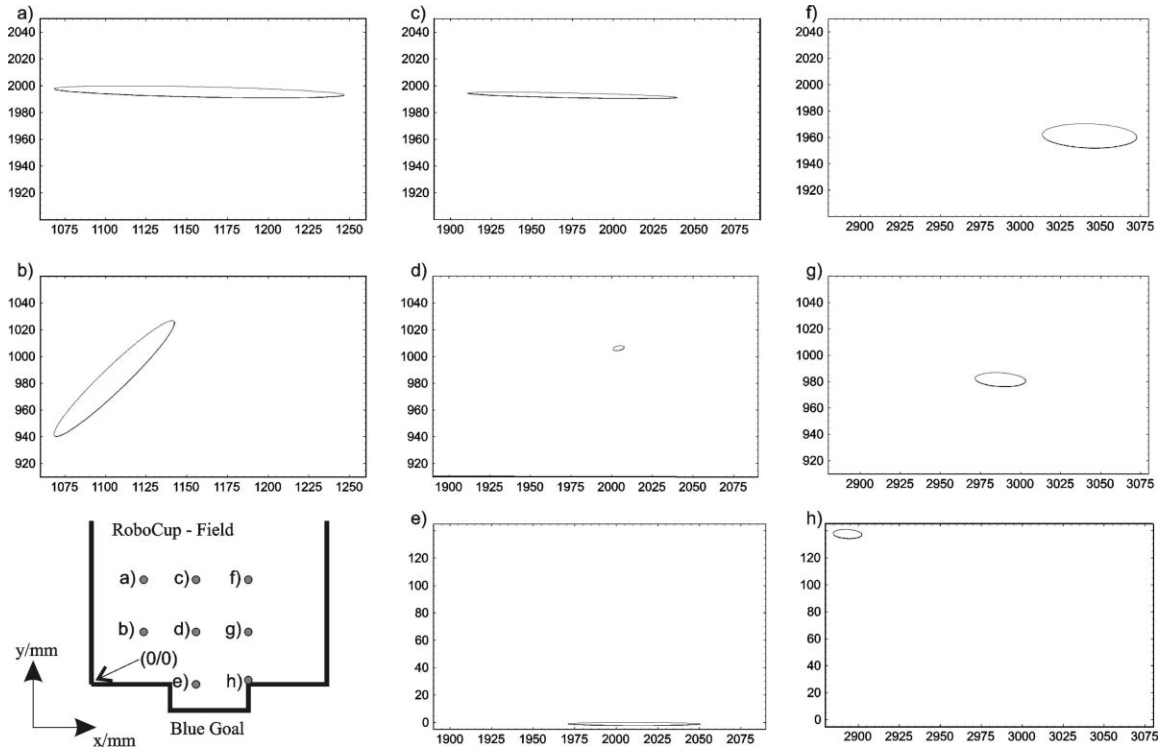


Fig. 10. Results for position at eight different locations within a RoboCup field.

for generating an appropriate mapping. If a robot for whose camera system the mapping is to be obtained is additionally equipped with a laser range finder, the data from the laser range finder can be used as distance reference values. The second method is used for a robot without a laser range finder. It is put at a well-defined position within a modeled environment. This permits the prediction of distances from the given position to objects (walls, goals) in the environment, which are then used as reference.

Self-localization. In the experiment (Fig. 10), the robot was placed in eight different positions (a–f) pointing to the blue goal. At each position 1000 position estimates were taken. The origin of the coordinate frame coincides with a corner of the RoboCup field which is defined by the intersection of the blue goal line and the left field boundary, with respect to the connecting line from the blue to the yellow goal. That means that the blue goal line runs from [1500, 0] to [3000, 0] in this coordinate frame since the experiments were carried out in a field setup corresponding to the RoboCup'98 rules. For each position, the mean and the covariance matrices were calculated over the 1000 position estimates. For the following it is assumed that the estimates can be modeled by a normal distribution. The ellipses in the figures represent a constant Mahalanobis distance from the center of these normal distributions. As can be seen from the results the described algorithm supplies sufficiently accurate position estimates within a certain area in front of a goal. That the width of the distributions in the x-direction is significantly larger than the width in the y-direction is a characteristic of the algorithm. As described in Section 2.1 this algorithm tries to fit a line to the set of points which are identified as representing the blue goal. The length of this line is often shorter than the goal width, due to the characteristics of the averaging process at the borders of the point set. Since there is a lack of other information about the alignment in the x-direction, the algorithm simply assumes that the midpoint of the fitted line coincides

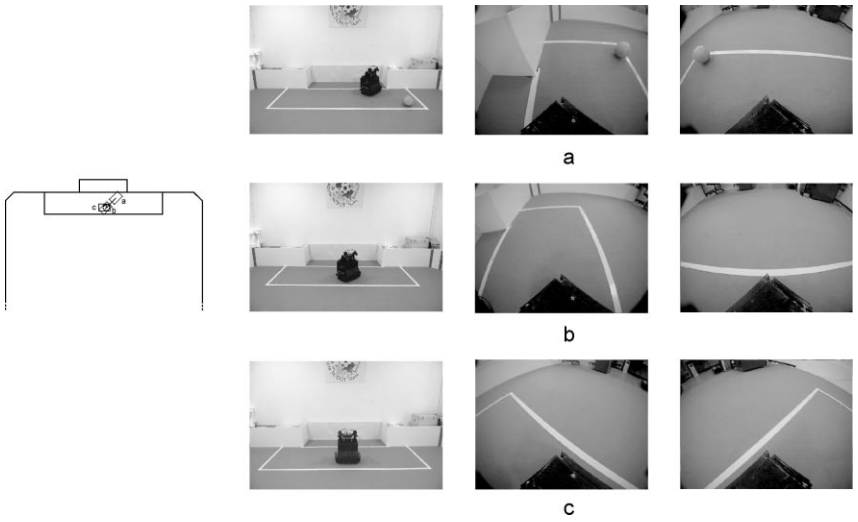


Fig. 11. Example of the goalie's maneuver. Left: scheme of the maneuver; right: three moments (a–c) of the maneuver sequence, as seen from outside and from the goalkeeper's left and right cameras.

with the midpoint of the goal line. This inaccuracy in the x-direction alignment leads to the observed asymmetric distributions.

The self-localization method described in Section 3 has been implemented and used on the robot goalkeepers (TinoZoff and Galavrón) that participated in RoboCup 1999 and 2000 as members of ART. [1]

In both robots, landmark detection is used not only for self-localization during "active" play situations (i.e., when the ball is approaching or is in the proximity of the goal), but also for repositioning, which may occur when the robot is not actively engaged in a playing situation. In such an "inactive" game situation, the goalkeeper first tries to locate the front line of the goalkeeper area. In the typical case the line is found and the robot position can be computed as described. The basic repositioning maneuver can be performed in this situation. It consists of three steps: firstly, a 45° movement towards or away from the line is performed, depending on the "vertical" offset with respect to the default position; secondly, once the right vertical position is reached, a rotation is performed to align the robot with the line. Finally, the horizontal offset from the default position is compensated by a sidewards movement which completes the maneuver (see Fig. 11).

When the goalkeeper has gone out of the goalkeeper area, so that the line is not visible, the other main landmarks of the RoboCup environment are used to roughly estimate the robot's rotation and horizontal displacement with respect to the default position. In particular, detecting the goalie's own goal is enough for this purpose. When the robot is out of its area without being rotated, its own goal is not visible, and the center line can be used, using geometrical reasoning in the same way as for the goalkeeper area front line, to understand if the robot is located to the right or to the left of its target position.

The results of this rather fast self-localization step allow the robot to decide in which direction it should head during the first segment of its backward movement towards the goal. Because of its kinematics, the robot cannot move straight forward or backwards; long forward and backward movements are obtained by means of subsequent pairs of short movements along directions which deviate alternatively 45° (or −45°) from the desired direction, with a "zig-zag" trajectory. Each segment of the "zig-zag" is in fact

[1] ART, Azzurra Robot Team, a joint effort research of the Universities of Genoa, Padova, Parma, Roma "La Sapienza" and Politecnico di Milano, supported by "Consorzio Padova Ricerche", is the Italian representative that takes part in the RoboCup F-2000 events (see http://robocup.ce.unipr.it for more details).

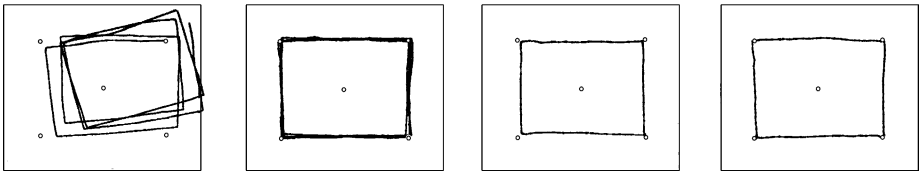


Fig. 12. Four rounds of the Sparrow-99 along a rectangular path in the RoboCup field (odometry path, corrected path using goal post features, distance features, and all features, respectively).

the result of a rotation, followed by a shift sidewards [1].

When the backward movement has made the goalkeeper area front line once more visible, the robot is in the same situation as in the basic case just described and the corresponding repositioning maneuver can be applied.

The MCL method was implemented and evaluated on a Sparrow-99 robot, a custom-built soccer robot equipped with a unidirectional camera system.

Experiment 1: Number of features. In this experiment, we show that the robot can localize robustly and that the accuracy can be improved by adding more features. A Sparrow-99 robot was placed in one corner of the field (the right top circle of Fig. 12). In order to have an accurate reference path, we moved the robot by hand along a rectangular trajectory indicated by the dots. The first image in Fig. 12 displays the odometry data when moving four rounds and shows the drift error that occurs. The second image displays the corrected trajectory which does not drift away. The third image displays the first round corrected by the localization algorithm using only the goal posts as features. In the fourth image we can see a more accurate path found using all three feature types.

Experiment 2: Number of samples. Implementing sample-based localization, it is important to have an idea of how many samples one needs. Obviously, a small number of samples is preferred, since the computational effort increases with the number of samples. On the other side, an appropriate number of samples is needed in order to achieve the desired accuracy. In this experiment, we moved four rounds exactly as in the previous experiment and used five different numbers of samples: 50, 100, 150, 1000, and 5000. In the left image of Fig. 13, the average localization errors of the different sample numbers are shown. As expected, the error decreases when more samples are used. On

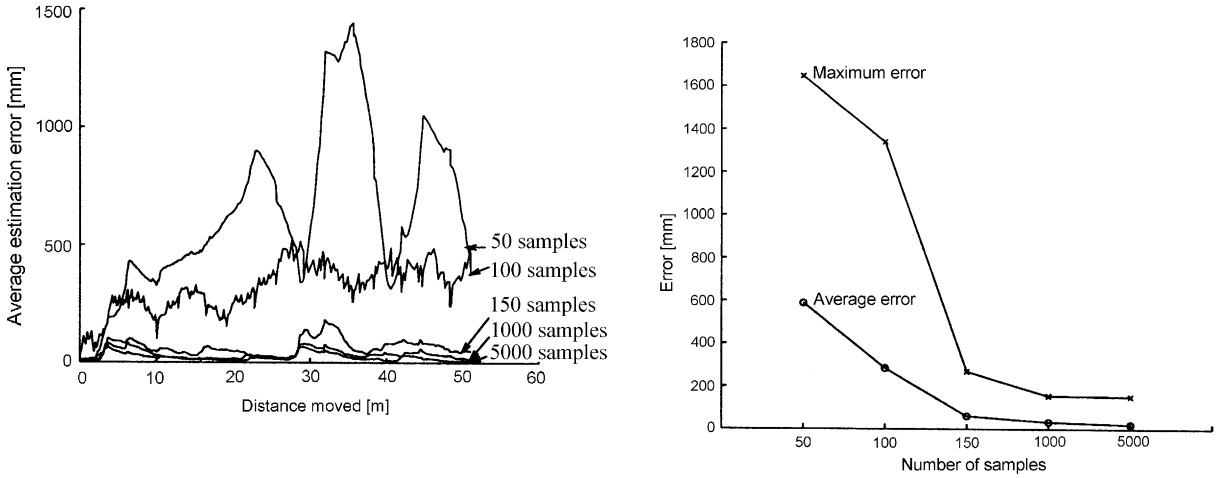


Fig. 13. Effects of using different sample numbers (50, 100, 150, 1000 and 5000 samples). Left: average localization errors. Right: average and maximum errors.

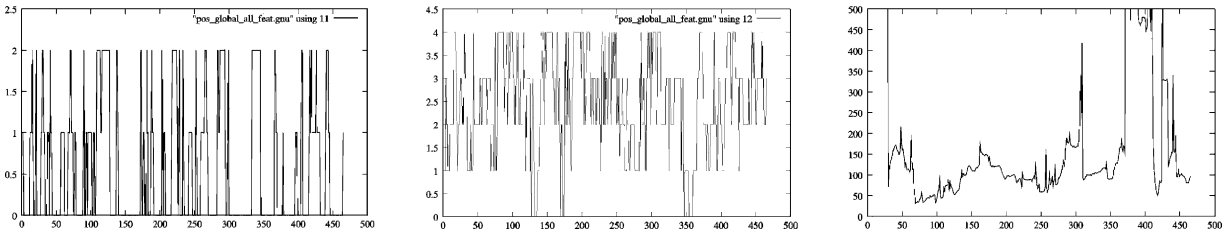


Fig. 14. Sparsity and sporadicality: number of landmark features (left) and distance features (middle) detected during a sample run in a real field, and the resulting standard deviations of samples in the sample set.

the other hand, the difference in accuracy does not increase much from 150 samples to over 1000–5000. The right image of Fig. 13 shows both the average error and the maximum error of the same five runs (50, 100, 150, 1000 and 5000 samples). Again, you can see that above 150 samples, the accuracy hardly increases.

Experiment 3: Sparsity and sporadicality. The third experiment illustrates the performance of vision-based MCL under playing conditions, i.e., how MCL deals with sparse and sporadic features (see Fig. 14). The figures show that even over extended periods (30 cycles) where no landmark features and very few or no distance features can be detected, the robot can still maintain a sufficiently precise location estimate. This is in accordance with the common wisdom that odometry is reliable for narrowly limited motion sequences. MCL ensures that the robot quickly recoups and computes better position estimates if features can be detected again.

6. Conclusions

The RoboCup competition has proved to be an interesting research platform for the study of autonomous intelligent systems which can interact with the real physical world. In particular, RoboCup is particularly challenging for vision systems, that need to integrate efficiency and accuracy to be able to deal with the real-time requirements of the dynamic and rapidly changing environment in which the competition takes place. All three systems described in the paper were extensively tested on the field during the latest editions of RoboCup.

The Attempto team Tübingen [29] implemented the algorithm described in Section 2 for the goalkeeper robot of the team. The images from a omnidirectional camera were utilized for a distance measurement to certain points on the field. This information was used with self-localization algorithms which were originally developed for laser range finders. Even with the worse resolution of the distance measurement with respect to a laser range finder, the position estimates of the algorithm were sufficiently accurate.

The goalkeeper of the ART team relied on the landmark-based localization described in Section 3. The robustness and reliability of the system played a key role in the performance of the goalkeeper that allowed the team to reach the second place in RoboCup 1999 and in the EuroRoboCup 2000 tournament.

The Ulm Sparrows team used the Monte Carlo approach for vision-based localization of a player. The onboard camera was used for localization purposes in addition to object recognition tasks. As a consequence, sensor input to update the robot's belief about its position was low-dimensional and sporadic. Nevertheless, the experimental evaluation demonstrated that MCL works well even under these restrictive conditions. Even with a small number of detected features, the localization results are usable. However, by increasing the number of visual features the accuracy enhances significantly.

Although the three approaches to vision-based localization were designed for the RoboCup competition, they can be adapted to other semi-structured indoor or outdoor environments quite easily. In indoor environments several natural landmarks that can be used for localization or navigation can be found; the same happens in some outdoor environments in which landmarks are commonly present. The geometric reasoning on images acquired by the binocular vision system described in Section 3 can be applied, for example, to automatic lane detection in computer-assisted vehicle driving, or to

wall-following in corridor-like indoor environments. In those cases an angular disparity between the scenes observed by the two cameras can be recorded between the two edges of a road or road lane, or between the borders between the floor and the walls, respectively.

Vision-based systems, despite not usually being the top-performers among robot sensor systems, are generally applicable and very flexible. All three approaches demonstrate that vision-based systems such as the ones described in the paper can in most cases be the only source of information for self-localization and navigation of autonomous robots. This is particularly important when the use of other more accurate but less flexible sensors imposes restrictive requirements that are not met by the environment in which robots operate.

Acknowledgements

The authors from the University of Ulm would like to thank all the student members of the RoboCup team *The Ulm Sparrows* for their great commitment and support, and G. Palm, the head of the department for his encouragement, many valuable discussions, and for providing the necessary resources. Financial support for the Ulm team was provided by the University administration and the University Society Ulm.

The projects developed at the University of Parma that are described in this paper have been partially funded by ENEA (Italian Organization for Alternative Energy) under the SENSI (Intelligent Sensors) grant, ASI (Italian Space Agency) under the "Hybrid Vision System for Long Range Rovering" grant and CNR (Italian National Research Council) under the "ART — Azzurra Robot Team" grant.

References

[1] G. Adorni, S. Cagnoni, M. Mordonini, Landmark-based robot self-localization: A case study for the RoboCup goal-keeper, in: Proceedings of the International Conference on Information Intelligence and Systems, IEEE Computer Society Press, Silver Spring, MD, 1999, pp. 164–171.

[2] G. Adorni, M. Gori, M. Mordonini, Just-in-time sign recognition in image sequences, Journal of Real-Time Imaging 5 (1999) 95–107.

[3] N. Aihara, H. Iwasa, N. Yokoya, H. Takemura, Memory-based self-localization using omnidirectional images, in: Proceedings of the 14th International Conference on Pattern Recognition, IEEE Computer Society Press, Silver Spring, MD, 1998, pp. 1799–1803.

[4] Y. Arai, T. Fuji, H. Asama, H. Kaetsu, I. Endo, Local communication-based navigation in a multirobot environment, Advanced Robotics 13 (3) (1999) 233–234.

[5] S. Atiya, G.D. Hager, Real-time vision-based robot localization, IEEE Transactions on Robotics and Automation 9 (6) (1993) 352–358.

[6] J. Borenstein, H.R. Everett, L. Feng, Navigating Mobile Robots: Sensors and Techniques, A.K. Peters Ltd., Wellesley, MA, 1996.

[7] J. Borenstein, H.R. Everett, D. Wehe, Mobile robots positioning: Sensors and techniques, Robotics and Autonomous Systems 14 (4) (1997) 231–249.

[8] D. Burschka, S.A. Blum, Identification of 3D reference structures for video-based localization, in: Computer Vision — ACCV'98, Third Asian Conference on Computer Vision. Proceedings Vol. 1, Springer, Berlin, 1997, pp. 128–135.

[9] K. Chung, Markov Chains with Stationary Transition Probabilities, Springer, Berlin, 1960.

[10] T. Dean, M. Boddy, An analysis of time-dependent planning, in: Proceedings of AAAI-88, St. Paul, MN AAAI Press, Menlo Park, CA, 1988, pp. 49–54.

[11] F. Dellaert, W. Burgard, D. Fox, S. Thrun, Using the condensation algorithm for robust, vision-based mobile robot localization, in: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 1999, pp. 588–594.

[12] H.S. Dulimaria, A.K.K. Jain, Mobile robot localization in indoor environment, Pattern Recognition 30 (1) (1997) 99–111.

[13] T. Einsele, Real-time self-localization in unknown indoor environment using a panorama laser range finder, in: Proceedings of the 1997 IEEE/RSJ International Conference on Intelligent Robot and Systems. Innovative Robotics for Real-world Applications, IROS'97, Vol. 2, IEEE Press, New York, 1997, pp. 697–702.

[14] S. Enderle, M. Ritter, D. Fox, S. Sablatnög, G. Kraetzschmar, G. Palm, Soccer-robot localization using sporadic visual features, in: Proceedings of the IAS-6 International Conference on Intelligent Autonomous Systems, Venice, Italy, 2000, pp. 959–966.

[15] D. Fox, Markov localization: A probabilistic framework for mobile robot localization and navigation, Ph.D. Thesis, University of Bonn, Bonn, Germany, December 1998.

[16] D. Fox, W. Burgard, F. Dellaert, S. Thrun, Monte Carlo localization: Efficient position estimation for mobile robots, in: Proceedings of AAAI'99, Orlando, FL, AAAI Press, Menlo Park, CA, 1999, pp. 343–349.

[17] M.O. Franz, B. Schölkopf, H.A. Mallot, H.H. Bülthoff, Where did I take that snapshot? Scene-based homing by image matching, Biological Cybernetics 79 (1999) 191–202.

[18] N. Guil, J. Villalba, E.L. Zapata, A fast Hough transform for segment detection, IEEE Transactions on Image Processing 4 (11) (1995) 1541–1548.

[19] J.S. Gutmann, W. Hatzack, I. Herrmann, B. Nebel, F. Rittinger, A. Topor, T. Weigel, Reliable self-localization,

multirobot sensor integration, accurate path-planning and basic soccer skills: playing an effective game of robotic soccer, in: Proceedings of the Ninth International Conference on Advanced Robotics ICAR'99, Japan Robot Association, 1999, pp. 289–296.
[20] M. Kabuka, B. Hussain, Real-time system for accurate three-dimensional determination and verification, IEEE Transactions on Robotics and Automation 6 (1) (1990) 3143.
[21] K. Kanatani, N. Ohta, Optimal robot self-localization and accuracy bounds, IEICE Transactions on Information and Systems E82-D (2) (1999) 447–452.
[22] H. Kitano, M. Asada, Y. Kuniyoshi, I. Noda, E. Osawa, H. Matsubara, RoboCup: A challenge problem for AI, AI Magazine 18 (1) (1997) 73–85.
[23] D.H. Kite, M. Magee, Determining the 3D position and orientation of a robot camera using 2D monocular vision, Pattern Recognition 23 (8) (1990) 819–831.
[24] M. Magee, J.K. Aggarwal, Robot self-localization using visual reasoning relative to a single target object, Pattern Recognition 28 (2) (1995) 125–134.
[25] P.S. Maybeck, The Kalman Filter: An Introduction to Concepts, Springer, Berlin, 1990, pp. 194–204.
[26] L. Moreno, J.M. Armingol, A. de-la-Escalera, M.A. Salichs, Global integration of ultrasonic sensors information in mobile robot localization, in: Proceedings of the Ninth International Conference on Advanced Robotics, 1999, pp. 283–298.
[27] E.M. Mouaddib, B. Marhic, Geometrical matching for mobile robot localization, IEEE Transactions on Robotics and Automation 16 (5) (2000) 542–552.
[28] S.B. Nickerson, P. Jasiobedzki, D. Wilkes, M. Jenkin, E. Milios, J. Tsotsos, A. Jepson, O.N. Bains, The ARK project: Autonomous mobile robots for known industrial environments, Robotics and Autonomous Systems 25 (1–2) (1998) 83–104.
[29] M. Plagge, R. Günther, J. Ihlenburg, D. Jung, A. Zell, The Attempto RoboCup robot team, in: M. Veloso, E. Pagello, H. Kitano (Eds.), RoboCup-99: Robot Soccer World Cup III, Lecture Notes in Artificial Intelligence, Vol. 1856, Springer, Berlin, 2000, pp. 424–433.
[30] J. Reuter, Mobile robot self-localization using PDAB, in: Proceedings of the 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation, Vol. 4, IEEE Press, New York, 2000, pp. 3512–3518.
[31] A.F.M. Smith, A.E. Gelfand, Bayesian statistics without tears: A sampling–resampling perspective, American Statistician 46 (2) (1992) 84–88.
[32] S. Suzuki, T. Kato, M. Asada, An application of vision-based learning in RoboCup for a real robot with an omnidirectional vision system and the team description of Osaka University "trackies", in: M. Asada, H. Kitano (Eds.), RoboCup-98: Robot Soccer World Cup II, Proceedings of the Second RoboCup Workshop, Lecture Notes in Artificial Intelligence, Vol. 1604, Springer, Berlin, pp. 316–325.
[33] G. Weiss, E. von Puttkammer, Intelligent Autonomous Systems, Map Based on Laserscans without Geometric Interpretation, IOS Press, Amsterdam, 1995, pp. 403–407 (Chapter A).



Giovanni Adorni, formerly with the University of Parma, is a Full Professor at the Department of Communications, Computer and System Science of the University of Genoa. His research interests include artificial intelligence, computer vision, robotics and soft computing. He is a member of AAAI, IAS and AI∗IA.



Stefano Cagnoni is an Assistant Professor at the Department of Computer Engineering of the University of Parma. His research interests include computer vision, pattern recognition, robotics and soft computing. He is a member of IEEE-CS, IAPR and AI∗IA.



Stefan Enderle is a Ph.D. candidate at the Neural Information Processing Department at the University of Ulm. His research interests include sensor interpretation and fusion, map building and self-localization in robotics, and software development for autonomous mobile robots.



Gerhard K. Kraetzschmar is a Senior Research Scientist and Assistant Professor at the Neural Information Processing Department at the University of Ulm. His research interests include autonomous mobile robots, neurosymbolic integration, learning and memory, and robot control and agent architectures. He is a member of IEEE, AAAI, ACM, IAS, and GI.



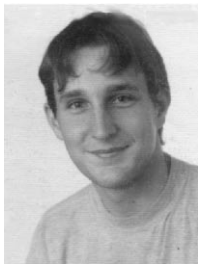
Monica Mordonini is an Assistant Professor at the Department of Computer Engineering of the University of Parma. Her research interests include computer vision, robotics and soft computing.



Michael Plagge is a Ph.D. candidate at the Department of Computer Architecture at the University of Tübingen. His research interests include multisensor fusion, fast and robust world modeling and tracking of dynamic objects with autonomous mobile robots.



Marcus Ritter was a M.S. student in Computer Science at the University of Ulm. He is now an Engineer at Wonderbits.



Stefan Sablatnög is a Ph.D. candidate at the Neural Information Processing Department at the University of Ulm. His research interests include spatial representation and reasoning in robotics and software development for autonomous mobile robots. He is a member of IEEE.



Andreas Zell is a Full Professor at the Department of Computer Architecture at the University of Tübingen. His research interests cover a wide range of fields in robotics, neural networks, evolutionary computation and genetic programming, bioinformatics and multimedia-aided learning and teaching. He is a member of IEEE and GI.